

Drafting for class template argument deduction issues

Discussion

This paper is intended to resolve

- The direction on wrapping vs. copying from EWG on Monday in Kona
- Handling of implicitly declared constructors ([discussed here](#))
- Access control ([discussed here](#))
- Redeclaration ([discussed here](#))
- Default-initialized declarations ([discussed here](#))
- Deduction in new-expressions
- Deduction in variable declarations other than simple-declarations

Proposed Resolution

☐ Hide deleted text

Change 5.3.4¶2:

...

If a placeholder type (7.1.7.4) appears in the type-specifier-seq of a new-type-id or type-id of a new-expression, the ~~new-expression shall contain a new-initializer of the form~~

~~(assignment-expression)~~

The allocated type is deduced ~~from the new-initializer~~ as follows: Let ~~e be the assignment-expression in the new-initializer~~ I be the new-initializer, if any, and T be the new-type-id or type-id of the new-expression, then the allocated type is the type deduced for the variable x in the invented declaration (7.1.7.4):

T x ~~(e)~~ I;

[Example:

```
new auto(1); // allocated type is int
auto x = new auto(â€™â€™); // allocated type is char, x is of type char*
```

```
template<class T> struct A { A(T,T); };
auto y = new A{1,2}; // allocated type is A<int>
```

— end example]

Change 7.1.7.2¶2:

... A type-specifier of the form `typenameopt nested-name-specifieropt template-name` is a placeholder for a deduced class type ~~and shall appear only as a decl-specifier in the decl-specifier-seq of a simple declaration (7.1.7.5) or as the simple-type-specifier in an explicit type conversion (functional notation) (5.2.3)~~. The template-name shall name a class template that is not an injected-class-name. ...

Change 7.1.7.4¶4-5:

The type of a variable declared using `auto` or `decltype(auto)` is deduced from its initializer. This use is allowed ~~when declaring variables in a block (6.3), in namespace scope (3.3.6), and in an init-statement (Clause 6)~~ in an initializing declaration (8.6) of a variable. `auto` or `decltype(auto)` shall appear as one of the decl-specifiers in the decl-specifier-seq and the decl-specifier-seq shall be followed by one or more ~~init~~ declarators, each of which shall have a non-empty initializer. In an initializer of the form

(expression-list)

the expression-list shall be a single assignment-expression. [Example: ... —end example]

A placeholder type can also be used ~~in declaring a variable in the condition of a selection statement (6.4) or an~~

iteration statement (6.5); in the type-specifier-seq in the new-type-id or type-id of a new-expression (5.3.4); in a for-range-declaration, in declaring a static data member with a brace-or-equal-initializer that appears within the member-specification of a class definition (9.2.3.2); and as a decl-specifier of the parameter-declarationTM's decl-specifier-seq in a template-parameter (14.1).

Change 7.1.7.5:

If a placeholder for a deduced class type appears as a decl-specifier in the decl-specifier-seq of a simple- an initializing declaration (8.6) of a variable, the ~~init-declarator of that declaration shall be of the form~~

~~declarator-id attribute-specifier-seq opt initializer~~

The placeholder is replaced by the return type of the function selected by overload resolution for class template deduction (13.3.1.8). If the decl-specifier-seq is followed by an init-declarator-list or member-declarator-list containing contains more than one ~~init-~~ declarator, the type that replaces the placeholder shall be the same in each deduction.

A placeholder for a deduced class type can also be used in the type-specifier-seq in the new-type-id or type-id of a new-expression (5.3.4), or as the simple-type-specifier in an explicit type conversion (functional notation) (5.2.3). A placeholder for a deduced class type shall not appear in any other context. [Example:

```
template<class T> struct container {
    container(T t) {}
    template<class Iter> container(Iter beg, Iter end);
};
template<class Iter>
container(Iter b, Iter e) -> container<typename std::iterator_traits<Iter>::value_type>;
std::vector<double> v = { /* ... */ };

container c(7); // OK, deduces int for T
auto d = container(v.begin(), v.end()); // OK, deduces double for T
container e{5, 6}; // error, int is not an iterator
```

— end example]

Add to the end of 8.6:

A declaration that specifies the initialization of a variable, whether from an explicit initializer or by default-initialization, is called the *initializing declaration* of that variable. [Note: In most cases this is the defining declaration (3.1) of the variable, but the initializing declaration of a non-inline static data member (9.2.3.2) might be the declaration within the class definition and not the definition at namespace scope. — end note]

Change 13.3.1.8:

A set of functions and function templates is formed comprising:

- For each constructor of the primary class template designated by the template-name, if the template is defined, a function template with the following properties:
 - The template parameters are the template parameters of the class template followed by the template parameters (including default template arguments) of the constructor, if any.
 - The types of the function parameters are those of the constructor.
 - The return type is the class template specialization designated by the template-name and template arguments corresponding to the template parameters obtained from the class template.
- If the primary class template C is not defined or does not declare any constructors, an additional function template derived as above from a hypothetical constructor C().
- An additional function template derived as above from a hypothetical constructor C(C), called the *copy deduction candidate*.
- For each deduction-guide, a function or function template with the following properties:
 - The template parameters, if any, and function parameters are those of the deduction-guide.
 - The return type is the simple-template-id of the deduction-guide.

Initialization and overload resolution are performed as described in 8.6 and 13.3.1.3, 13.3.1.4, or 13.3.1.7 (as appropriate for the type of initialization performed) for an object of a hypothetical class type, where the selected functions and function templates are considered to be the constructors of that class type for the purpose of forming an overload set, and the initializer is provided by the context in which class template argument deduction was performed. Each such notional constructor is considered to be explicit if the function or function template was generated from a constructor or deduction-guide that was declared explicit. All such notional constructors are considered to be public members of the hypothetical class type.

In 13.3.3[1, move the deduction guide bullet after partial ordering:

- ...
- **F1 is generated from a deduction-guide (13.3.1.8) and F2 is not [Example:**

```
template <class T> struct A {  
    A(T, int*); // #1  
    A(A<T>&, int*); // #2  
    enum { value };  
};  
  
template <class T, int N = T::value> A(T&&, int*) -> A<T>; // #3  
  
A a{1, 0}; // uses #1 to deduce A<int> and initializes with #1  
A b{a, 0}; // uses #3 (not #2) to deduce A<A<int>&> and initializes with #1
```

—end example] or, if not that,

- F1 is not a function template specialization and F2 is a function template specialization, or, if not that,
- F1 and F2 are function template specializations, and the function template for F1 is more specialized than the template for F2 according to the partial ordering rules described in 14.5.6.2, **or, if not that,**
- **F1 is generated from a deduction-guide (13.3.1.8) and F2 is not, or, if not that,**
- **F1 is the copy deduction candidate (13.3.1.8) and F2 is not, or, if not that,**
- **F1 is generated from a non-template constructor and F2 is generated from a constructor template. [Example:**

```
template <class T> struct A {  
    using value_type = T;  
    A(value_type); // #1  
    A(const A&); // #2  
    A(T, T, int); // #3  
    template<class U> A(int, T, U); // #4  
}; // A(A); #5, the copy deduction candidate  
  
A x (1, 2, 3); // uses #3, generated from a non-template constructor  
  
template <class T> A(T) -> A<T>; // #6, less specialized than #5  
  
A a (42); // uses #6 to deduce A<int> and #1 to initialize  
A b = a; // uses #5 to deduce A<int> and #2 to initialize  
  
template <class T> A(A<T>) -> A<A<T>>; // #7, as specialized as #5  
  
A b2 = a; // uses #7 to deduce A<A<int>> and #1 to initialize
```

—end example]

Change 14.9:

The same restrictions apply to the parameter-declaration-clause of a deduction guide as in a function declaration (8.3.5). The simple-template-id shall name a class template specialization. The template-name shall be the same identifier as the template-name of the simple-template-id. A deduction-guide shall be declared in the same scope as the corresponding class template **and, for a member class template, with the same access. Two deduction guide declarations in the same translation unit for the same class template shall not have equivalent parameter-declaration-clauses.**